ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÔN: TƯ DUY TÍNH TOÁN

ĐỀ TÀI: NHẬT KÝ Y TẾ

Sinh viên thực hiện:

Nguyễn Đặng Khôi Nguyên (24120394)
Trịnh Vỹ Triết (24120474)
Trần Thị Lợi (24120370)
Cái Lâm Trường (24120480)
Lê Nguyễn Thùy Linh (24120085)

Giáo viên hướng dẫn:

Đậu Ngọc Hà Dương

Tp. Hồ Chí Minh, tháng 3 năm 2026

Mục lục

1 THÔNG TIN NHÓM **2**

2 TỔNG QUAN VẤN ĐỀ ĐỒ ÁN GIẢI QUYẾT **2**
2.1 Bối cảnh và Vấn đề chung 2
2.2 Giải pháp đề xuất: Hệ sinh thái Web và App 2

3 CÁC VẤN ĐỀ ĐƯỢC THẢO LUẬN THEO 4 BƯỚC CỦA TƯ DUY TÍNH TOÁN **3**
3.1 Vấn đề 1: Thông tin y tế rời rạc, khó truy cập trong khẩn cấp 3
3.2 Vấn đề 2: Bác sĩ thiếu dữ liệu sức khỏe dài hạn của bệnh nhân 3
3.3 Vấn đề 3: Người dùng lười nhập liệu thói quen hàng ngày 4
3.4 Vấn đề 4: Hiển thị dữ liệu phù hợp theo từng ngữ cảnh sử dụng 4
3.5 Vấn đề 5: Thiết kế Module SOS trong tình huống khẩn cấp 5
3.6 Vấn đề 6: Truy cập nhanh khẩn cấp mà vẫn đảm bảo tính riêng tư 5
3.7 Vấn đề 7: Bảo mật thông tin các tầng khác cho người dùng 6
3.8 Vấn đề 8: Tính sẵn sàng cao và khả năng lưu trữ lâu dài 6
3.9 Vấn đề 9: Duy trì phiên đăng nhập an toàn cho ứng dụng y tế 7

4 BÁO CÁO TIẾN ĐỘ **7**
4.1 Backend 7
4.1.1 Kiến trúc tổng quan hệ thống 7
4.1.2 Thiết lập hạ tầng và môi trường phát triển 7
4.1.3 Cơ sở dữ liệu và ORM 8
4.1.4 Hệ thống Middleware 9
4.1.5 Hệ thống xác thực (Authentication) 9
4.1.6 Bảo mật dữ liệu nhạy cảm 10
4.1.7 Xử lý lỗi tập trung (Global Exception Handlers) 10
4.1.8 Schemas dùng chung (Shared Schemas) 11
4.1.9 Tổng kết tiến độ Backend 11
4.2 Frontend 11
4.2.1 Thiết kế UI/UX 11
4.2.2 Chuyển đổi thiết kế sang mã nguồn 12
4.2.3 Các màn hình chính được thiết kế (dự kiến) 12
4.2.4 Công nghệ Frontend dự kiến 12
4.2.5 Tổng kết tiến độ Frontend 12

5 CÔNG NGHỆ SỬ DỤNG (DỰ KIẾN) **13**
5.1 Backend 13
5.2 Frontend (Native App) 13
5.3 Database 13
5.4 Deployment 13

1 THÔNG TIN NHÓM

MSSV	Họ và Tên	Địa chỉ email	Ghi chú
24120394	Nguyễn Đặng Khôi Nguyên	24120394@student.hcmus.edu.vn	
24120474	Trịnh Vỹ Triết	24120474@student.hcmus.edu.vn	
24120370	Trần Thị Lợi	24120370@student.hcmus.edu.vn	
24120480	Cái Lâm Trường	24120480@student.hcmus.edu.vn	
24120085	Lê Nguyễn Thùy Linh	24120085@student.hcmus.edu.vn	

2 TỔNG QUAN VẤN ĐỀ ĐỒ ÁN GIẢI QUYẾT

2.1 Bối cảnh và Vấn đề chung

Trong nhiều tình huống khẩn cấp như tai nạn giao thông, đột quỵ hoặc ngất xỉu tại nơi công cộng, người gặp nạn thường không thể cung cấp thông tin y tế quan trọng cho nhân viên cấp cứu. Những thông tin như bệnh nền, dị ứng thuốc, thuốc đang sử dụng, nhóm máu hoặc thông tin người thân cần liên hệ thường chỉ được lưu trữ trong hồ sơ bệnh án tại bệnh viện hoặc do chính bệnh nhân ghi nhớ.

Trong các tình huống bất ngờ, việc thiếu các dữ liệu y tế quan trọng này có thể khiến nhân viên cấp cứu gặp khó khăn trong việc đánh giá tình trạng ban đầu của bệnh nhân và lựa chọn phương án xử lý phù hợp. Điều này có thể làm chậm quá trình cấp cứu hoặc dẫn đến những rủi ro trong việc điều trị, đặc biệt trong các trường hợp cần xử lý nhanh để đảm bảo *"thời gian vàng"* cứu sống bệnh nhân.

Bên cạnh đó, tại nhiều địa phương hiện nay vẫn chưa có một hệ thống phổ biến giúp lưu trữ và cung cấp nhanh các thông tin y tế cá nhân trong các tình huống khẩn cấp. Điều này khiến việc tiếp cận thông tin sức khỏe của người bệnh trong thời điểm cần thiết trở nên khó khăn.

2.2 Giải pháp đề xuất: Hệ sinh thái Web và App

Để giải quyết triệt để các vấn đề nêu trên, đồ án tập trung vào việc thiết kế và xây dựng một hệ sinh thái y tế toàn diện bao gồm cả nền tảng **Web** và **Ứng dụng di động (App)**. Sự kết hợp này nhằm tận dụng tối đa thế mạnh của từng nền tảng trong các ngữ cảnh sử dụng khác nhau:

- **Ứng dụng di động (Mobile App):** Đóng vai trò là vật bất ly thân của người dùng. App sẽ chịu trách nhiệm chính trong việc lưu trữ thẻ y tế thông minh (Offline-first), tích hợp các tiện ích cứu hộ khẩn cấp (SOS), và tự động thu thập các chỉ số sức khỏe hàng ngày (nhịp tim, bước chân). Đây là công cụ giúp người dùng làm chủ dữ liệu sức khỏe của mình mọi lúc, mọi nơi.

- **Nền tảng Web (Web App):** Được thiết kế dành riêng cho các nhân viên y tế và bác sĩ. Nền tảng này cho phép truy cập vào hồ sơ bệnh án chi tiết khi có sự đồng ý của bệnh nhân, hỗ trợ việc chẩn đoán dựa trên các biểu đồ xu hướng dữ liệu dài hạn mà App đã thu thập. Ngoài ra, Web cũng đóng vai trò là cổng quản trị trung tâm để quản lý danh tính và bảo mật toàn hệ thống.

3 CÁC VẤN ĐỀ ĐƯỢC THẢO LUẬN THEO 4 BƯỚC CỦA TƯ DUY TÍNH TOÁN

3.1 Vấn đề 1: Thông tin y tế rời rạc, khó truy cập trong khẩn cấp

Vấn đề: Thông tin y tế cá nhân (nhóm máu, tiền sử dị ứng, bệnh nền, thuốc đang sử dụng, thông tin người thân liên hệ) hiện nay tồn tại rời rạc ở nhiều nơi: sổ khám bệnh giấy, file PDF trên email, ứng dụng của từng bệnh viện, hoặc chỉ đơn giản là trong trí nhớ của người dùng.

- **Decomposition:** Thiếu dữ liệu người bệnh và không liên lạc được người thân. Khi ngất xỉu hoặc hoảng loạn, nạn nhân không thể nhớ, không thể tìm sổ, và không thể đọc mật khẩu mở app cho người cấp cứu.
- **Pattern Recognition:** Khi có sự cố khẩn cấp là bệnh nhân lại "kẹt" thông tin. Ta nhận ra nhu cầu bức thiết là cần một "hồ sơ trung tâm" duy nhất, luôn gắn liền với người bệnh (thông qua chiếc điện thoại di động).
- **Abstraction:** Thiết kế một ứng dụng đóng vai trò như một chiếc ví lưu giữ thông tin sức khỏe.
- **Algorithm Design:** Thiết kế luồng quản lý dữ liệu cơ bản của ứng dụng:
  - **Đầu vào:** Cung cấp giao diện để người dùng nhập thông tin cá nhân và bệnh án.
  - **Lưu trữ:** Ghi nhận và sở hữu khối dữ liệu đó tại hệ thống lưu trữ của ứng dụng.
  - **Đầu ra:** Trích xuất và hiển thị thành một Thẻ Y tế tổng hợp, sẵn sàng cho người dùng hoặc bác sĩ đọc ngay lập tức.

3.2 Vấn đề 2: Bác sĩ thiếu dữ liệu sức khỏe dài hạn của bệnh nhân

Vấn đề: Khi khám bệnh, bác sĩ thường thiếu dữ liệu nền dài hạn về sức khỏe của bệnh nhân. Các quyết định y khoa thường chỉ dựa vào các chỉ số đo được tại một thời điểm duy nhất (lúc ở phòng khám) và lời kể mơ hồ của bệnh nhân về thói quen sinh hoạt, dẫn đến việc khó phát hiện các dấu hiệu suy giảm sức khỏe từ từ.

- **Decomposition:** Ta chia nhu cầu dữ liệu chẩn đoán thành 2 phần: chỉ số tức thời tại phòng khám (ví dụ: đo huyết áp ngay lúc đó) và dữ liệu lịch sử kéo dài (ví dụ: nhịp tim trung bình trong 30 ngày qua, thói quen vận động). Sự thiếu hụt trầm trọng đang nằm ở phần dữ liệu lịch sử.
- **Pattern Recognition:** Bệnh lý thường không xuất hiện đột ngột mà là kết quả của một chuỗi biến đổi sinh lý kéo dài. Nhận ra mẫu ở đây là: Dữ liệu chuỗi thời gian mang lại giá trị chẩn đoán cao hơn rất nhiều so với dữ liệu tại một thời điểm.
- **Abstraction:** Mở rộng khả năng của ứng dụng từ việc chỉ "lưu thẻ tĩnh" sang việc lưu trữ liên tục các chỉ số sinh hiệu (nhịp tim, bước chân) và thói quen để tạo thành một "Bức tranh xu hướng".
- **Algorithm Design:** Thiết kế cấu trúc lưu trữ mở rộng cho hệ thống: Thay vì chỉ ghi đè một bản ghi duy nhất như Thẻ SOS, hệ thống sẽ tạo một cơ sở dữ liệu lưu trữ tình trạng gắn liền với mốc thời gian. Mỗi ngày trôi qua sẽ tạo ra một bản ghi mới chứa các trường: Ngày tháng, Nhịp tim, Bước chân, Triệu chứng cơ thể,...

3.3 Vấn đề 3: Người dùng lười nhập liệu thói quen hàng ngày

Vấn đề: Làm sao để có "Dữ liệu thói quen" khi người dùng quá lười nhập liệu mỗi ngày?

- **Decomposition:** Chia dữ liệu thói quen thành: Thứ máy tự đo được (nhịp tim, bước chân) và thứ con người phải tự cảm nhận (tâm trạng, đau nhức).
- **Pattern Recognition:** Hành vi lặp đi lặp lại: Người dùng cầm điện thoại/đeo đồng hồ thông minh cả ngày (máy luôn có data), nhưng rất lười mở app ra gõ chữ.
- **Abstraction:** Bỏ qua các form điền chữ dài dòng. Giữ lại 2 thứ cốt lõi: Lấy dữ liệu ngầm từ máy và bắt người dùng chọn 1 thanh đánh giá mức độ.
- **Algorithm Design:**
  - Cài đặt trên máy để tự động thu thập thông tin về nhịp tim, bước chân trung bình của một người mỗi ngày thông qua ứng dụng sức khỏe được cài đặt mặc định trên các dòng máy.
  - Khi có triệu chứng cần ghi lại, có một giao diện để người dùng nhập tên triệu chứng và nhập tình trạng hoặc chọn icon đánh giá mức độ của triệu chứng. Có một phần riêng nếu người dùng muốn ghi chú kỹ.

3.4 Vấn đề 4: Hiển thị dữ liệu phù hợp theo từng ngữ cảnh sử dụng

Vấn đề: Khi hệ thống thu thập thành công một lượng lớn dữ liệu (từ thông tin cơ bản, nhịp tim đến cảm xúc hàng ngày), việc hiển thị tất cả chúng trên cùng một giao diện sẽ không cần thiết. Người cấp cứu cần thông tin sinh tử trong vòng vài giây, trong khi bác sĩ phòng khám cần dữ liệu xu hướng trong vòng 5 phút. Việc trộn lẫn dữ liệu sẽ làm mất đi tính ứng dụng của hệ thống trong các tình huống thực tế.

- **Decomposition:** Ta tiến hành bóc tách toàn bộ "khối dữ liệu sức khỏe" và "ngữ cảnh sử dụng" thành các thành phần nhỏ để đối chiếu:
  - **Về dữ liệu:** Có nhóm máu, dị ứng, SĐT khẩn cấp, nhịp tim trung bình, bước chân, mức độ đau nhức, trạng thái tâm lý...
  - **Về ngữ cảnh:** Có tình huống cấp cứu khẩn cấp, tình huống khám bệnh, và tình huống tự theo dõi.
- **Pattern Recognition:** Khi đối chiếu các tập hợp trên, ta phát hiện ra một quy luật phân bổ rõ rệt về mức độ khẩn cấp và tính chất thay đổi của dữ liệu:
  - Dữ liệu ít khi thay đổi (tĩnh) + Cực kỳ quan trọng cho sự sống còn → Thường dùng trong Cấp cứu.
  - Dữ liệu thay đổi liên tục (động) + Đo lường khách quan → Thường dùng cho Bác sĩ phân tích bệnh lý.
  - Dữ liệu mang tính cảm nhận (chủ quan) → Thường dùng làm thông tin tham khảo phụ.
- **Abstraction:** Gạt bỏ đi các chi tiết cụ thể (không cần quan tâm dị ứng hải sản hay dị ứng thuốc lúc này). Khái quát hóa và định nghĩa khối dữ liệu thành 3 tầng thông tin cốt lõi dựa trên mức độ ưu tiên:
  - **Tầng Sinh tử (Cố định):** Quyết định tính mạng ngay lập tức.

  - **Tầng Biến thiên (Xu hướng):** Cung cấp bức tranh y khoa khách quan.
  - **Tầng Thói quen (Tâm trạng):** Cung cấp bối cảnh sinh hoạt chủ quan.

- **Algorithm Design:** Thiết kế thuật toán định tuyến hiển thị dựa trên việc gắn thẻ dữ liệu vào 3 tầng đã trừu tượng hóa ở trên:
  - **Nhánh 1:** Khi khởi chạy Module SOS, thuật toán chỉ truy xuất và hiển thị tập dữ liệu thuộc "Tầng Sinh tử" lên một giao diện thiết kế tối giản, font chữ lớn, bỏ qua mọi rào cản xác thực.
  - **Nhánh 2:** Khi khởi chạy Module Bác sĩ/Nhật ký, bên cạnh "Tầng Sinh tử", thuật toán sẽ tiến hành gọi dữ liệu từ "Tầng Biến thiên" và "Tầng Thói quen", sau đó đưa qua các hàm vẽ biểu đồ để tổng hợp thành Dashboard trực quan.

3.5 Vấn đề 5: Thiết kế Module SOS trong tình huống khẩn cấp

Vấn đề: Thiết kế Module SOS như thế nào trong tình huống khẩn cấp?

- **Decomposition:** Khó khăn của người bệnh lúc này thành các yếu tố vật lý và tâm lý: Thị giác (không đọc được chữ nhỏ), Vận động (tay run, chạm không chính xác), Nhận thức (não bộ đang đóng băng, không thể xử lý chuỗi logic phức tạp).
- **Pattern Recognition:** Khi hoảng loạn, cần thao tác đơn giản để gửi yêu cầu cấp cứu khẩn cấp.
- **Abstraction:** Thiết kế một nút bấm đơn giản, màu đỏ, đủ to để trường hợp khẩn cấp người dùng có thể dễ dàng ấn vào.
- **Algorithm Design:** Thiết kế một Shortcut đủ to, màu đỏ, để khi người dùng ấn vào, thông báo cấp cứu sẽ được gửi đi.

Vấn đề phụ: Người dùng ấn nhầm nút SOS

- **Decomposition:** Nút bấm to dễ bị cơ thể/ngón tay vô tình cọ trúng trong túi quần.
- **Pattern Recognition:** Hành động khẩn cấp cần sự xác nhận có chủ đích và thông báo kích hoạt.
- **Abstraction:** Trì hoãn gửi thông báo, cho phép rút lại trong thời gian ngắn.
- **Algorithm Design:** Nút SOS nằm trong giao diện Shortcut (giảm rủi ro ấn nhầm). Khi kích hoạt, điện thoại rung và phát âm thanh cảnh báo; người dùng có thể hủy yêu cầu trước khi hệ thống thực hiện cuộc gọi/tin nhắn khẩn cấp.

3.6 Vấn đề 6: Truy cập nhanh khẩn cấp mà vẫn đảm bảo tính riêng tư

Vấn đề: Truy cập nhanh dữ liệu sinh tử của người bị nạn mà không vi phạm quyền riêng tư các dữ liệu khác.

- **Decomposition:** Trong khẩn cấp, người hỗ trợ cần nắm "Tầng sinh tử" ngay lập tức khi nạn nhân mất ý thức. Các tầng dữ liệu khác phải được bảo mật.
- **Pattern Recognition:** Người dùng luôn mang theo điện thoại và CCCD. CCCD chỉ công khai thông tin định danh cơ bản, tương tự luồng cấp cứu chỉ nên trích xuất "Tầng sinh tử".

- **Abstraction:** Truy cập qua tra cứu số CCCD hoặc quét mã QR định danh in sẵn.
- **Algorithm Design:**
  - Người dùng cập nhật dữ liệu "Tầng sinh tử" và số CCCD lên hệ thống trung tâm.
  - Hệ thống xuất mã QR để người dùng dán sau điện thoại hoặc để trong ví.
  - **Truy xuất:** Quét mã QR hoặc nhập số CCCD vào cổng tra cứu khẩn cấp để nhận ngay thông tin nhóm máu, dị ứng, liên hệ khẩn cấp.

3.7 Vấn đề 7: Bảo mật thông tin các tầng khác cho người dùng

Vấn đề: Kiểm soát quyền truy cập dữ liệu nhạy cảm (Tầng biến thiên, Tầng thói quen) của bác sĩ.

- **Decomposition:** Dữ liệu này dùng trong thăm khám thường quy. Truy cập của bác sĩ phải là tạm thời và có sự đồng ý của bệnh nhân.
- **Pattern Recognition:** Dữ liệu riêng tư phải nằm dưới quyền kiểm soát tuyệt đối của bệnh nhân. Sự minh bạch đến từ việc cấp quyền dùng một lần và có lịch sử truy cập (Audit Log).
- **Abstraction:** Công cụ yêu cầu truy cập từ bác sĩ và xác nhận phê duyệt từ bệnh nhân với giới hạn thời gian.
- **Algorithm Design:**
  - Bác sĩ gửi yêu cầu cấp quyền qua hệ thống (sau khi tra cứu bệnh nhân).
  - Hệ thống gửi thông báo đẩy/OTP tới thiết bị bệnh nhân.
  - Bệnh nhân nhấn "Đồng ý", hệ thống mở khóa dữ liệu và kích hoạt Timer.
  - Hết thời gian hoặc bác sĩ đóng phiên, quyền truy cập tự động thu hồi. Hệ thống ghi lại lịch sử truy cập hiển thị cho bệnh nhân.

3.8 Vấn đề 8: Tính sẵn sàng cao và khả năng lưu trữ lâu dài

Vấn đề: Phân bổ luồng dữ liệu giữa Local và Cloud để đảm bảo tốc độ và dung lượng.

- **Decomposition:** Local cần thao tác nhanh, không phụ thuộc mạng nhưng dung lượng hạn chế. Cloud lưu trữ trọn đời và an toàn nhưng cần kết nối.
- **Pattern Recognition:** Local làm bộ đệm ghi nhận tức thời. Cloud là kho lưu trữ chính. Đồng bộ ngầm diễn ra khi có mạng.
- **Abstraction:** Mô hình "Đồng bộ ngầm - Dọn dẹp tự động".
- **Algorithm Design:**
  - Dữ liệu mới ghi vào Local với cờ [Chờ đồng bộ].
  - Tiến trình ngầm đẩy dữ liệu lên Cloud khi có Internet. Nhận ACK từ Cloud thì đổi cờ thành [Đã đồng bộ].
  - Thuật toán quét Local hàng ngày: Xóa dữ liệu [Đã đồng bộ] cũ hơn 30 ngày để giải phóng bộ nhớ.
  - **Truy xuất:** 30 ngày gần nhất lấy từ Local (offline), dữ liệu cũ hơn tải từ Cloud.

3.9 Vấn đề 9: Duy trì phiên đăng nhập an toàn cho ứng dụng y tế

Vấn đề: Làm sao để duy trì phiên đăng nhập thuận tiện cho người dùng nhưng vẫn đảm bảo tính bảo mật cao của dữ liệu y tế, tránh rủi ro khi phiên đăng nhập bị kéo dài vô thời hạn hoặc bị đánh cắp?

- **Decomposition:** Chia nhỏ cơ chế xác thực thành các thành phần: Access Token (quyền truy cập ngắn hạn), Refresh Token (quyền cấp mới token), thời gian sống của token (TTL), và cơ chế phát hiện xâm nhập (đánh cắp token).
- **Pattern Recognition:** Nhận diện sự khác biệt về yêu cầu bảo mật: Các ứng dụng mạng xã hội ưu tiên trải nghiệm người dùng nên duy trì phiên đăng nhập hàng tháng. Tuy nhiên, ứng dụng y tế chứa dữ liệu nhạy cảm nên cần áp dụng mô hình phiên đăng nhập ngắn hạn để giảm thiểu thời gian.
- **Abstraction:** Lựa chọn thời gian sống (TTL) của Access Token hợp lý và xây dựng cơ chế xác thực, thu hồi phiên đăng nhập (Session Management) một cách chặt chẽ.
- **Algorithm Design:**
  - Thiết lập Access Token có TTL ngắn (khoảng 15 phút) trên hệ thống xác thực (Supabase).
  - **Cơ chế làm mới:** Khi Access Token hết hạn, ứng dụng tự động sử dụng Refresh Token để đổi lấy một cặp Access Token và Refresh Token mới.
  - **Phát hiện xâm nhập (Token Rotation):** Nếu một Refresh Token cũ được sử dụng lại (dấu hiệu của việc tin tặc đánh cắp và tái sử dụng), hệ thống sẽ lập tức vô hiệu hóa (Hard Delete) toàn bộ phiên đăng nhập của người dùng đó.
  - **Phản ứng:** Yêu cầu người dùng đăng nhập lại từ đầu và gửi thông báo cảnh báo bảo mật về hành vi truy cập bất thường.

4 BÁO CÁO TIẾN ĐỘ

Phần này tổng hợp chi tiết các công việc đã được triển khai trên hệ thống dự án "Nhật ký Y tế", tính đến thời điểm báo cáo. Toàn bộ mã nguồn được quản lý trên Git với nhánh `dev` (nhánh phát triển ổn định).

4.1 Backend

4.1.1 Kiến trúc tổng quan hệ thống

Backend được xây dựng theo kiến trúc **Domain-Driven Design (DDD)**, tổ chức mã nguồn thành các module nghiệp vụ độc lập. Mỗi module nghiệp vụ tuân theo cấu trúc chuẩn gồm: `models.py` (ORM), `schemas.py` (Pydantic), `service.py` (logic nghiệp vụ), và `router.py` (API endpoints).

4.1.2 Thiết lập hạ tầng và môi trường phát triển

Container hóa với Docker Hệ thống đã được container hóa hoàn chỉnh bằng Docker để đảm bảo môi trường nhất quán giữa các thành viên. Dockerfile sử dụng image `python:3.11-slim`, cài đặt các thư viện hệ thống cần thiết (`gcc`, `libpq-dev`), và chạy ứng dụng qua **Uvicorn** với chế độ hot-reload. File `docker-compose.yml` định nghĩa service `api` với cấu hình DNS public (8.8.8.8, 1.1.1.1) để đảm bảo kết nối ổn định đến Supabase Cloud.

Quản lý cấu hình Module `app/core/config.py` sử dụng **Pydantic Settings** để quản lý tập trung các biến môi trường, bao gồm:

- Thông tin kết nối Supabase (URL, Anon Key, Service Role Key).
- Chuỗi kết nối CSDL PostgreSQL (scheme `postgresql+asyncpg`).
- Cấu hình bảo mật: JWT Secret, thuật toán mã hóa (HS256).

Các thư viện chính Dự án sử dụng các thư viện Python quan trọng sau:

Thư viện	**Mục đích**
FastAPI	Framework API hiệu năng cao, hỗ trợ async
SQLAlchemy (asyncio)	ORM bất đồng bộ cho PostgreSQL
Alembic	Quản lý migration cơ sở dữ liệu
asyncpg	Driver PostgreSQL bất đồng bộ
Supabase SDK	Tích hợp Auth, Storage, Realtime
python-jose	Giải mã và xác minh JWT
Pydantic Settings	Quản lý cấu hình từ biến môi trường

Bảng 1: Danh sách các thư viện chính của Backend

4.1.3 Cơ sở dữ liệu và ORM

Mô hình Hybrid: SQLAlchemy + Supabase Hệ thống áp dụng kiến trúc **Hybrid**, trong đó:

- **SQLAlchemy AsyncSession:** Xử lý mọi thao tác CRUD trên PostgreSQL thông qua ORM bất đồng bộ.
- **Supabase Client:** Chỉ sử dụng cho các dịch vụ đặc thù: Authentication (đăng ký/đăng nhập qua phone), Storage (upload file), và Realtime (thông báo đẩy).

Định nghĩa Models (ORM) Hai bảng chính đã được định nghĩa đầy đủ:

- **Bảng `profiles`:** Lưu trữ thông tin người dùng bao gồm: họ tên, vai trò (user/doctor/admin), giới tính, ngày sinh, nhóm máu, tiền sử dị ứng, liên hệ khẩn cấp, cài đặt quyền riêng tư (JSONB). Các trường nhạy cảm như số điện thoại và CCCD được mã hóa bằng `pgcrypto`.
- **Bảng `doctors`:** Lưu thông tin bổ sung cho vai trò bác sĩ: chuyên khoa, số giấy phép hành nghề, bệnh viện, URL chứng chỉ, trạng thái xác minh (pending/approved/rejected), và thông tin admin duyệt.

Các bảng sử dụng cơ chế **Soft Delete** (cột `deleted_at`) thay vì xóa vĩnh viễn, đảm bảo khả năng truy vết và khôi phục dữ liệu.

Alembic Migrations Hệ thống quản lý phiên bản CSDL bằng **Alembic**. File migration đầu tiên (`dee690ba0a46_initial_schema.py`) đã được tạo, định nghĩa schema ban đầu cho toàn bộ hệ thống. Alembic được cấu hình để tự động phát hiện thay đổi từ các model SQLAlchemy và sinh migration tương ứng.

4.1.4 Hệ thống Middleware

RLS Middleware (Row Level Security) Middleware RLS (`app/middlewares/rls.py`) đảm nhiệm vai trò bảo mật dữ liệu ở tầng cơ sở dữ liệu. Luồng xử lý:

1. Trích xuất JWT token từ header `Authorization`.
2. Giải mã token bằng thư viện `python-jose` với secret key của Supabase.
3. Chuyển đổi payload thành định dạng claims chuẩn Supabase (role, sub, email, metadata).
4. Gắn claims vào `request.state` để truyền xuống tầng database.
5. Trong `get_db()`, claims được inject vào PostgreSQL session qua lệnh `SET LOCAL` để kích hoạt RLS policies.

Logging Middleware Middleware Logging (`app/middlewares/logging.py`) cung cấp khả năng theo dõi và truy vết mọi request/response:

- Tạo mã `request_id` duy nhất (UUID 8 ký tự) cho mỗi request đến.
- Ghi log thông tin request (method, path) và response (status code, thời gian xử lý tính bằng ms).
- Đính kèm `X-Request-ID` vào header response để phía client có thể truy vết.

4.1.5 Hệ thống xác thực (Authentication)

Module xác thực (`app/modules/auth/`) đã được triển khai hoàn chỉnh trên nhánh `feature/auth`, bao gồm:

Schemas xác thực Định nghĩa các Pydantic models cho toàn bộ luồng Auth:

- **LoginRequest / LoginResponse:** Xác thực bằng số điện thoại + mật khẩu. Response trả về `access_token` (JWT) và thông tin cơ bản của người dùng (id, role).
- **RegisterRequest:** Đăng ký tài khoản người dùng thường với các trường: số điện thoại, mật khẩu (tối thiểu 8 ký tự), họ tên, giới tính, ngày sinh.
- **RegisterDoctorRequest:** Mở rộng đăng ký cho bác sĩ, bổ sung: số CCCD (12 ký tự), số giấy phép hành nghề, chuyên khoa, bệnh viện.
- **SessionResponse / SessionListResponse:** Quản lý phiên đăng nhập của người dùng.

Tất cả schemas đều sử dụng **Field Validation** nghiêm ngặt: regex cho số điện thoại, `min_length`/`max_leng` cho mật khẩu và họ tên.

Service xác thực (AuthService) Lớp `AuthService` triển khai 5 phương thức nghiệp vụ chính:

1. **login():** Xác thực qua Supabase Auth, sau đó truy vấn trực tiếp PostgreSQL để lấy role thực tế từ bảng `profiles` (bypass Supabase SDK để hỗ trợ role nâng cao).
2. **register():** Tạo tài khoản trên Supabase Auth, sau đó INSERT profile vào CSDL với số điện thoại được mã hóa bằng `pgp_sym_encrypt()`.
3. **register_doctor():** Luồng đăng ký 3 bước: tạo Auth account → INSERT profile (mã hóa phone + CCCD) → INSERT doctor info với trạng thái `pending_verification`.
4. **log_out():** Hủy phiên đăng nhập trên Supabase Auth.
5. **list_session():** Truy vấn trực tiếp bảng `auth.sessions` để liệt kê các phiên đăng nhập đang hoạt động.

Mọi thao tác đều được bọc trong try/except với cơ chế `rollback()` khi xảy ra lỗi, đảm bảo tính toàn vẹn dữ liệu.

4.1.6 Bảo mật dữ liệu nhạy cảm

Mã hóa tầng cơ sở dữ liệu (pgcrypto) Hệ thống sử dụng extension **pgcrypto** của PostgreSQL để mã hóa các trường dữ liệu nhạy cảm trực tiếp trong CSDL:

- **Trường được mã hóa:** Số điện thoại (`phone_encrypted`), số CCCD (`cccd_encrypted`).
- **Phương thức:** Sử dụng hàm `pgp_sym_encrypt()` với khóa mã hóa được inject qua biến session `app.encryption_key`.
- **Giải mã:** Chỉ khi cần đọc, sử dụng `pgp_sym_decrypt()` với cùng khóa.

File SQL thiết lập (`supabase/policies/001_pgcrypto_setup.sql`) đã được tạo sẵn để kích hoạt extension trên Supabase.

Phân quyền và kiểm soát truy cập Module `app/shared/dependencies.py` cung cấp hệ thống phân quyền 3 lớp:

1. **get_supabase_client():** Khởi tạo Supabase Client an toàn với xử lý lỗi kết nối.
2. **get_current_user():** Trích xuất thông tin người dùng từ JWT claims đã được RLS Middleware xử lý. Trả về lỗi 401 nếu token không hợp lệ.
3. **require_role():** Dependency kiểm tra phân quyền theo role. Ví dụ: `Depends(require_role(["doctor", "admin"]))` sẽ chặn mọi request không có quyền tương ứng (lỗi 403).

4.1.7 Xử lý lỗi tập trung (Global Exception Handlers)

Module `app/core/exceptions.py` thiết lập hệ thống xử lý lỗi tập trung cho toàn bộ ứng dụng, đảm bảo mọi response lỗi đều tuân theo format chuẩn `ErrorResponse` (gồm `error_code`, `message`, `request_id`):

- **HTTP Exception Handler:** Bắt mọi lỗi HTTP (401, 403, 404, ...) và log cảnh báo kèm `request_id`.
- **Validation Exception Handler:** Bắt lỗi validation dữ liệu đầu vào (thiếu field, sai kiểu), trả về thông báo rõ ràng cho client.
- **Unhandled Exception Handler:** Bắt mọi lỗi không lường trước (500), log đầy đủ stack trace để debug nhưng chỉ trả về thông báo chung cho client nhằm tránh lộ thông tin hệ thống.

4.1.8 Schemas dùng chung (Shared Schemas)

Module `app/shared/schemas.py` định nghĩa các response schema chuẩn hóa được sử dụng xuyên suốt toàn bộ API:

- **ErrorResponse:** Format lỗi chuẩn với mã lỗi, thông báo, và mã truy vết request.
- **PaginatedResponse[T]:** Schema phân trang Generic, hỗ trợ phân trang cho mọi danh sách dữ liệu (items, total, page, limit).
- **MessageResponse:** Format trả về tin nhắn thành công đơn giản.

4.1.9 Tổng kết tiến độ Backend

Hạng mục	Trạng thái	Ghi chú
Kiến trúc DDD & cấu trúc thư mục	✓	11 module nghiệp vụ đã được khởi tạo
Docker & Docker Compose	✓	Môi trường dev hoạt động ổn định
SQLAlchemy ORM & Alembic	✓	Migration ban đầu đã tạo
Middleware RLS	✓	Inject JWT claims vào PostgreSQL session
Middleware Logging	✓	Request tracing với request_id
Exception Handlers	✓	3 handler xử lý lỗi tập trung
Module Auth (schemas + service)	✓	Login, Register, Register Doctor, Logout, Sessions
Mã hóa pgcrypto	✓	Mã hóa SĐT và CCCD tại tầng DB
Phân quyền (RBAC)	✓	3 lớp: auth check, role check, RLS
Auth Router (API endpoints)	Chưa	Cần tạo router kết nối service → endpoint
Các module nghiệp vụ khác	Chưa	diaries, emergency, health_metrics, ...
Unit Tests	Chưa	Cần bổ sung test cho auth service

Bảng 2: Bảng tổng kết tiến độ Backend

4.2 Frontend

Phần Frontend của dự án hiện đang trong giai đoạn thiết kế giao diện và xây dựng nguyên mẫu (prototyping), chưa tiến hành lập trình. Các công việc đã và đang thực hiện bao gồm:

4.2.1 Thiết kế UI/UX

Toàn bộ giao diện người dùng được thiết kế trên nền tảng **Figma** — công cụ thiết kế cộng tác trực tuyến cho phép nhiều thành viên cùng làm việc đồng thời trên cùng một bản thiết kế. Quy trình thiết kế tuân theo các bước:

1. **Nghiên cứu người dùng:** Xác định các nhóm người dùng chính (bệnh nhân, bác sĩ, nhân viên cấp cứu) và nhu cầu đặc thù của từng nhóm trong các ngữ cảnh sử dụng khác nhau (khẩn cấp, khám bệnh, tự theo dõi).
2. **Wireframe:** Phác thảo bố cục cơ bản cho các màn hình chính trên Figma, tập trung vào luồng thao tác (user flow) trước khi đầu tư vào chi tiết thẩm mỹ.
3. **Prototype tương tác:** Tạo nguyên mẫu có khả năng tương tác trên Figma để mô phỏng trải nghiệm thực tế trước khi chuyển sang giai đoạn lập trình.

4.2.2 Chuyển đổi thiết kế sang mã nguồn

Dự án dự kiến sử dụng **Stitch** — công cụ hỗ trợ chuyển đổi bản thiết kế từ Figma sang mã nguồn React Native một cách tự động. Stitch giúp:

- Xuất các thành phần giao diện từ Figma thành code React tái sử dụng được.
- Giảm đáng kể thời gian chuyển đổi từ thiết kế sang code, đảm bảo tính đồng nhất giữa bản thiết kế và sản phẩm thực tế.
- Cho phép đội phát triển tập trung vào logic nghiệp vụ thay vì phải dựng giao diện thủ công từ đầu.

4.2.3 Các màn hình chính được thiết kế (dự kiến)

Dựa trên phân tích các vấn đề tư duy tính toán ở Mục 3, các nhóm màn hình chính bao gồm:

- **Nhóm Xác thực:** Đăng nhập, Đăng ký (người dùng thường và bác sĩ).
- **Nhóm Thẻ Y tế (Tầng Sinh tử):** Hiển thị thông tin nhóm máu, dị ứng, liên hệ khẩn cấp với font chữ lớn, giao diện tối giản.
- **Nhóm SOS:** Nút SOS kích thước lớn, màu đỏ nổi bật; màn hình đếm ngược trước khi gửi tín hiệu cấp cứu.
- **Nhóm Nhật ký:** Giao diện ghi chép triệu chứng nhanh với icon đánh giá mức độ, biểu đồ xu hướng sức khỏe.
- **Nhóm Bác sĩ (Web):** Dashboard hiển thị dữ liệu bệnh nhân, biểu đồ chỉ số sinh hiệu dài hạn.

4.2.4 Công nghệ Frontend dự kiến

Công nghệ	Mục đích
React Native	Phát triển ứng dụng di động đa nền tảng (iOS, Android)
ReactJS	Phát triển giao diện Web dành cho bác sĩ và quản trị
TypeScript	Ngôn ngữ chính, đảm bảo type-safety
Zustand	Quản lý trạng thái ứng dụng (State Management)
SQLite	Lưu trữ dữ liệu ngoại tuyến trên thiết bị di động
Figma	Thiết kế giao diện và tạo prototype tương tác
Stitch	Chuyển đổi thiết kế Figma sang code React Native

Bảng 3: Công nghệ Frontend dự kiến sử dụng

4.2.5 Tổng kết tiến độ Frontend

Hạng mục	Trạng thái	Ghi chú
Nghiên cứu người dùng	✓	Xác định 3 nhóm người dùng chính
Wireframe các màn hình	Đang thực hiện	Thiết kế trên Figma
Prototype tương tác	Chưa	Sẽ tạo sau khi hoàn thành wireframe
Cài đặt project React	Chưa	Chờ hoàn thành thiết kế
Tích hợp Stitch	Chưa	Chuyển đổi Figma → code
Kết nối API Backend	Chưa	Chờ Backend hoàn thành Router

Bảng 4: Bảng tổng kết tiến độ Frontend

5 CÔNG NGHỆ SỬ DỤNG (DỰ KIẾN)

Dưới đây là mô tả các công nghệ dự kiến sử dụng cho dự án Nhật ký y tế:

5.1 Backend

- **Ngôn ngữ:** Python (3.10+).
- **Framework: FastAPI**.
- **Bảo mật (Security):** OAuth2 kết hợp JWT (JSON Web Token), mã hóa mật khẩu bằng bcrypt.
- **Kiểm định dữ liệu (Validation):** Pydantic.
- **Kiến trúc:** RESTful API.

5.2 Frontend (Native App)

- **Ngôn ngữ chính:** TypeScript / JavaScript (ES6+).
- **Framework/Library: React Native và ReactJS**.
- **Lưu trữ ngoại tuyến (Local DB):** SQLite.
- **Quản lý trạng thái (State Management):** Zustand.
- **Giao diện (UI Framework):** React Native Paper hoặc NativeWind.
- **Thiết kế:** UI tối giản, thân thiện, nút bấm to rõ để dễ thao tác trong tình huống khẩn cấp.

5.3 Database

- **Cơ sở dữ liệu:** PostgreSQL.
- **ORM (Object-Relational Mapping):** SQLAlchemy.
- **Cloud Database:** Supabase.

5.4 Deployment

- **Containerization: Docker**.
- **Orchestration:** Docker Compose.
- **Cloud Platform:** AWS (Amazon Web Services) hoặc Google Cloud Platform.